

UFUG 2602

Merge Sort

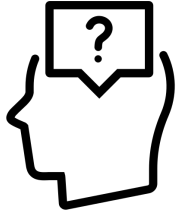
Tengfei Chang, Jia Li, Xihu Zheng, Zijun Gong

IoT, DSA, INTR Thrusts
Hong Kong University of Science and Technology (Guangzhou)

Divide & Conquer

- **Divide & Conquer** is an approach that solves a big problem by dividing the big problem into small problems and solve the small problems, which solves the big problem.

Divide & Conquer



- One example: **Moving House**



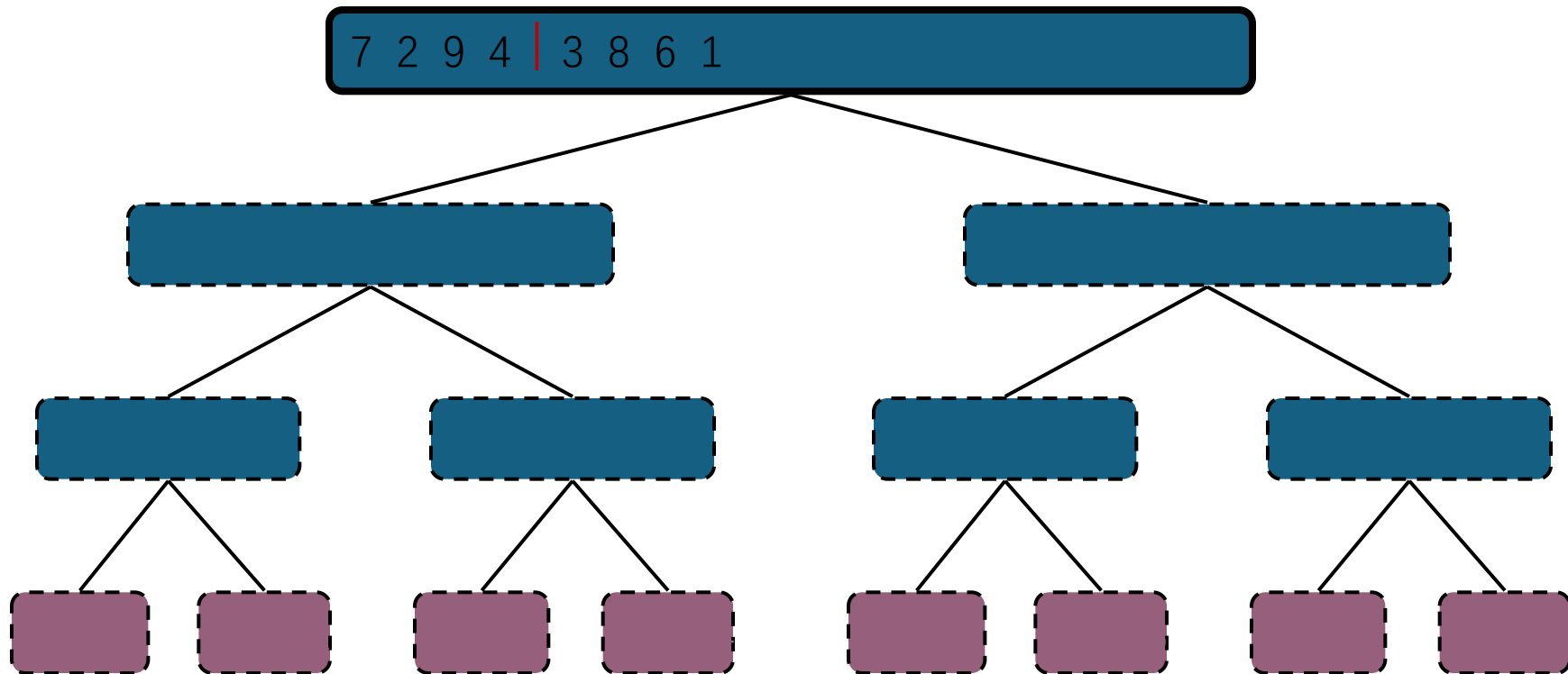
1. Divide the task of moving into smaller tasks (e.g., packing kitchen items, packing bedroom items, transportation, etc.).
2. Work together with family or friends by assigning each person to be responsible for a specific area.
3. Finally, move all belongings to the new house and organize them.

Merge Sort (Divide & Conquer)

- Consider a n -element sequence.
- Divide the n -element sequence into two smaller sequences.
 - The first has the first $\lfloor n/2 \rfloor$ elements.
 - The second has the remaining elements.
- Sort each of the two recursively.
- Consider both subproblems solved and merge the two halves of the array into the final sorted sequence (details later).

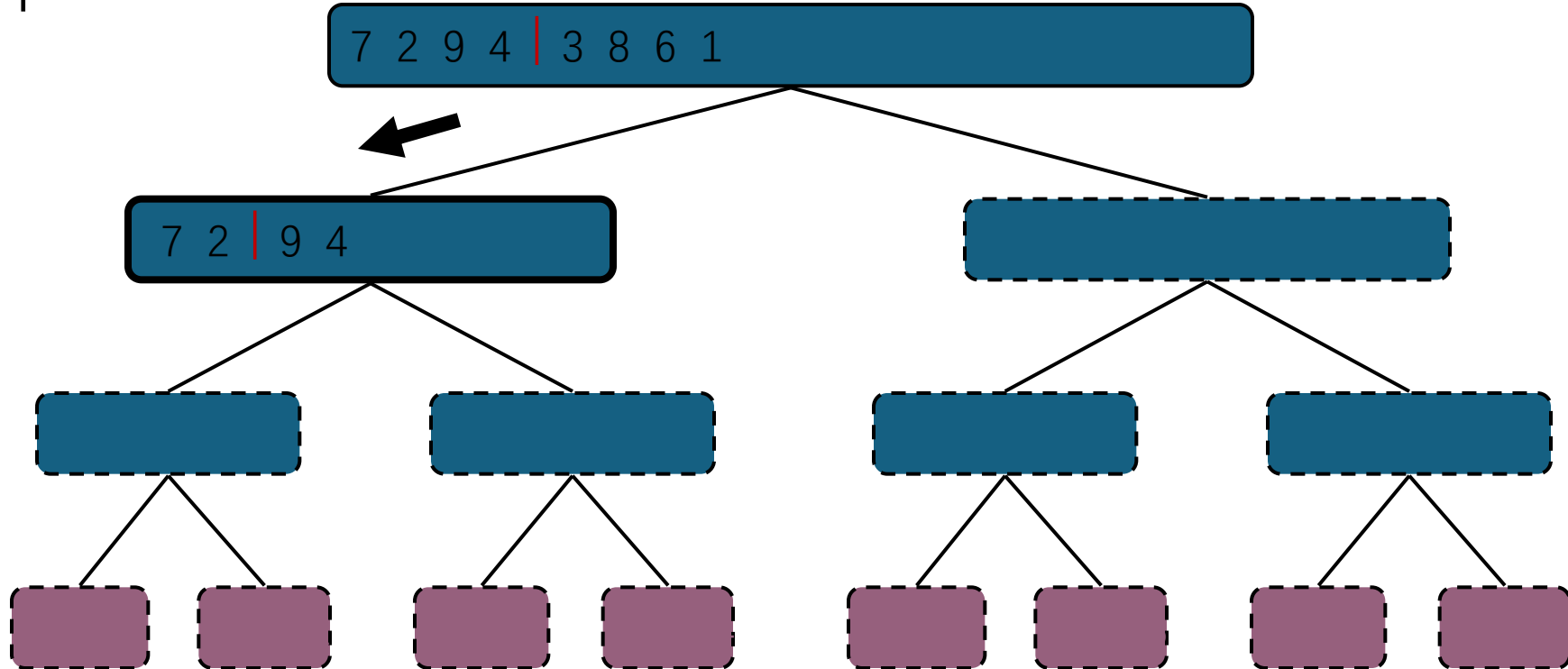
An Example (1)

- Partition the initial sequence into two subsequences.



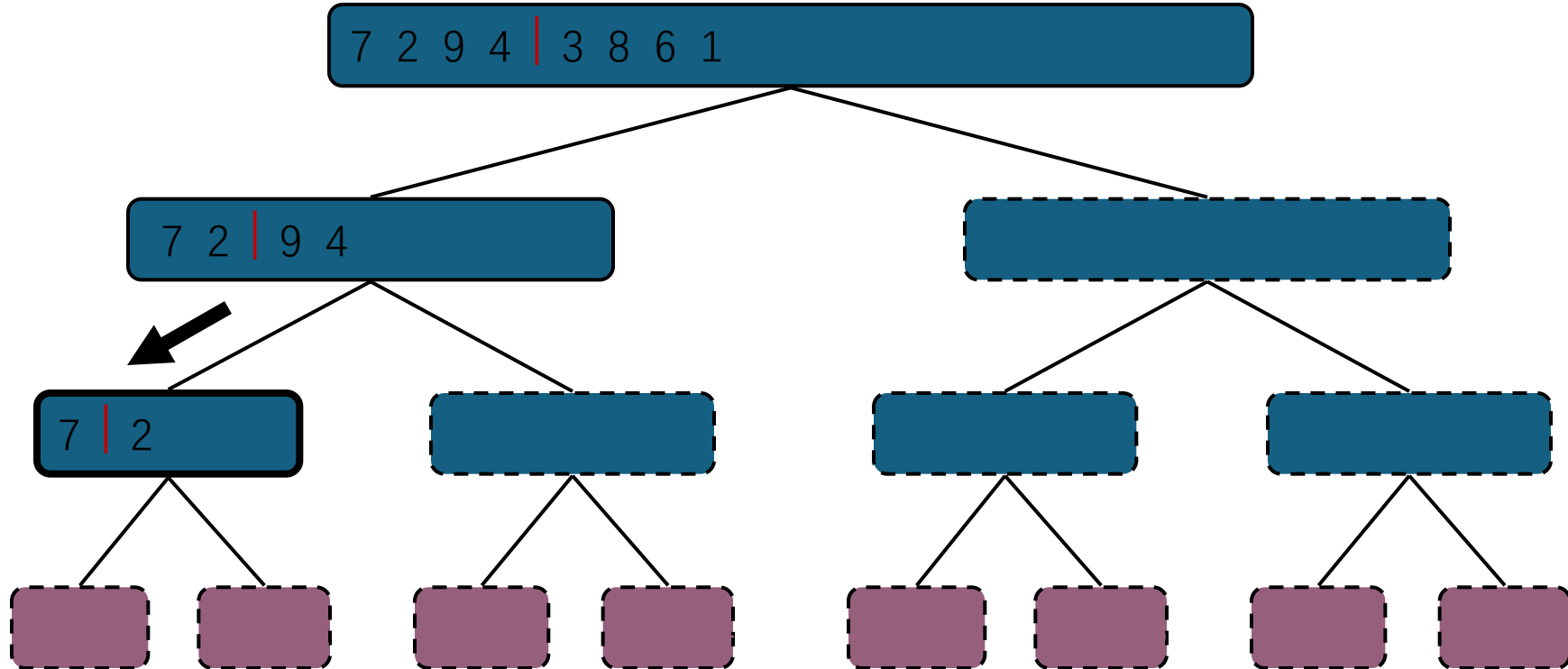
An Example (2)

- Recursive call, partition the first sequence into another two subsequences



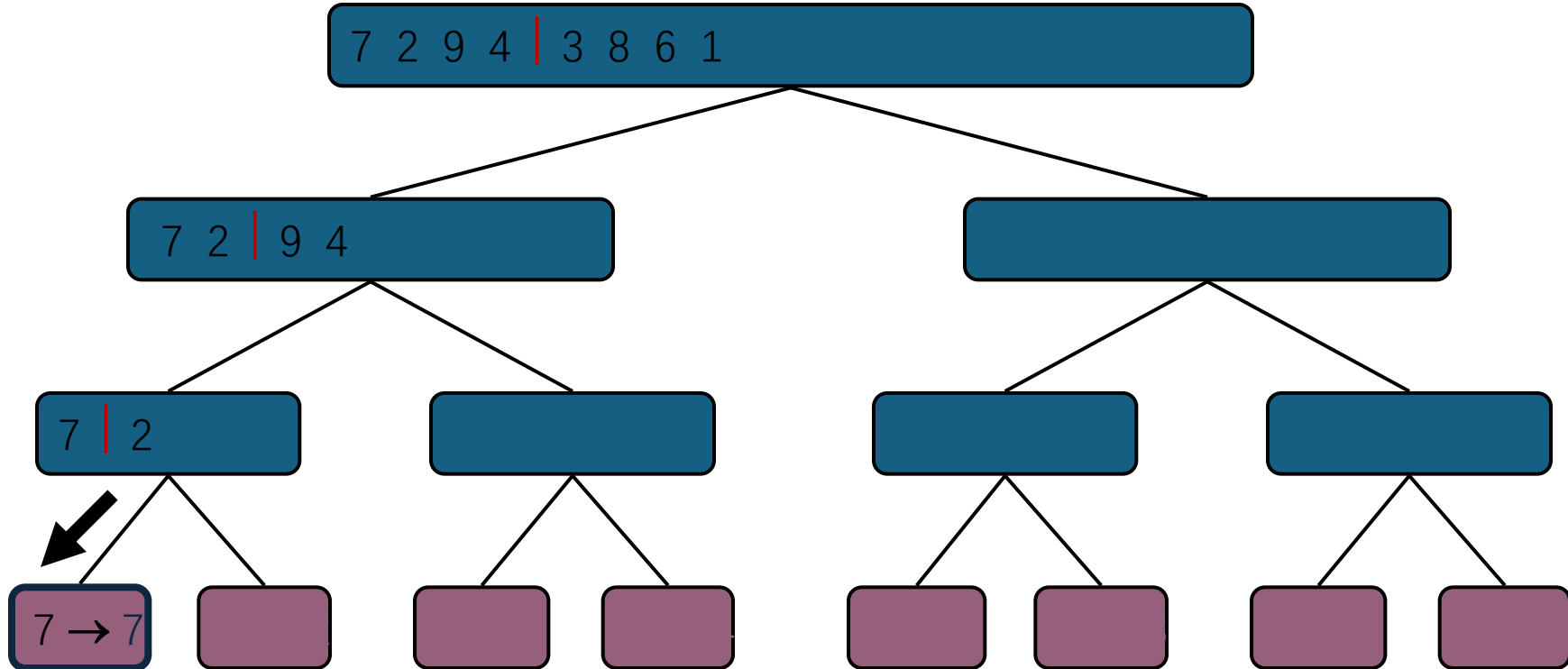
An Example (3)

- Recursive call, partition



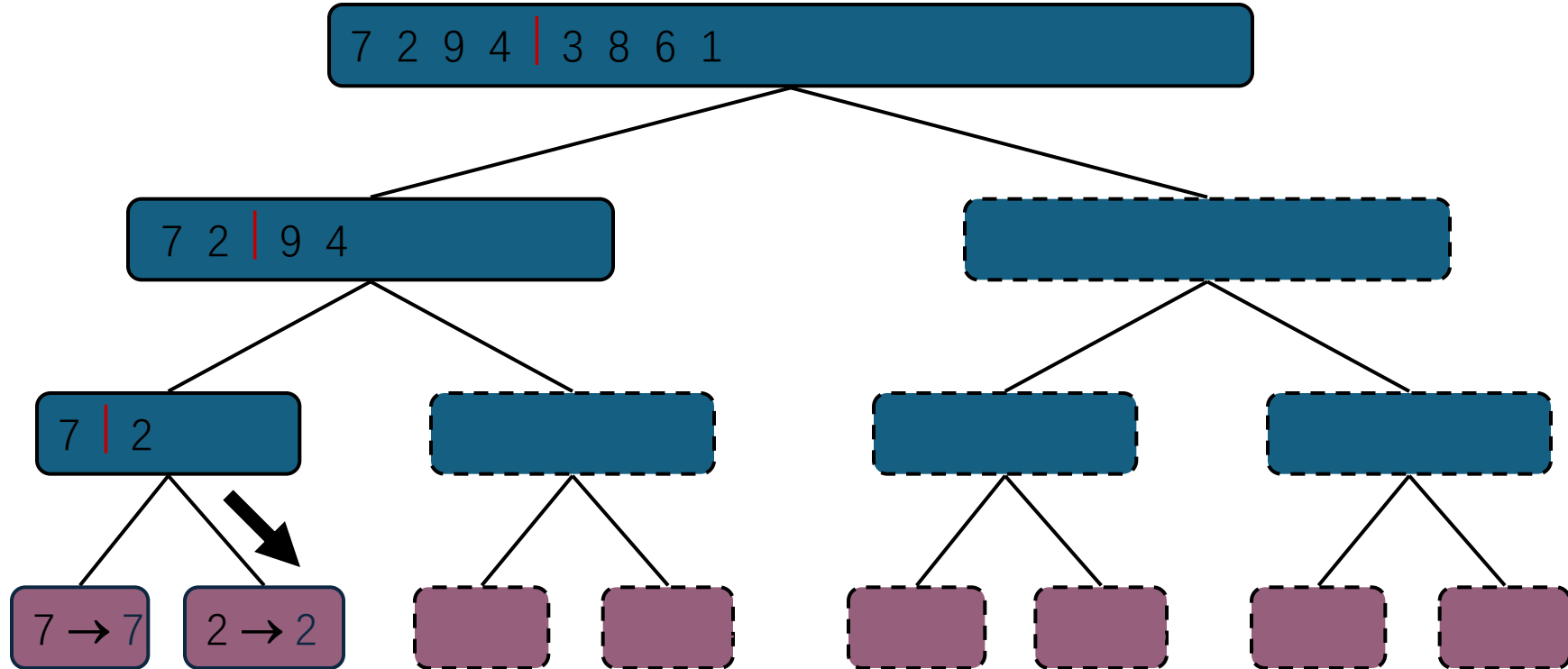
An Example (4)

- Recursive call, base case



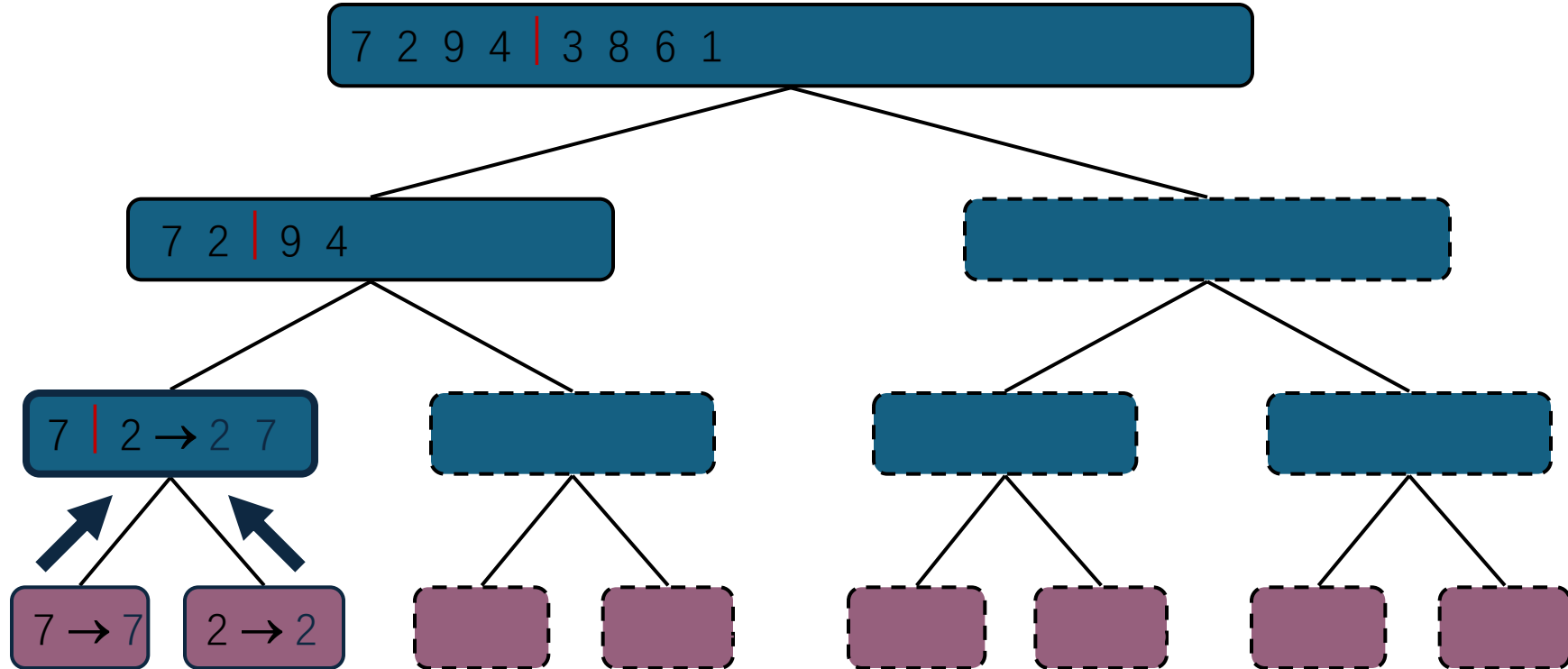
An Example (5)

- Recursive call, base case



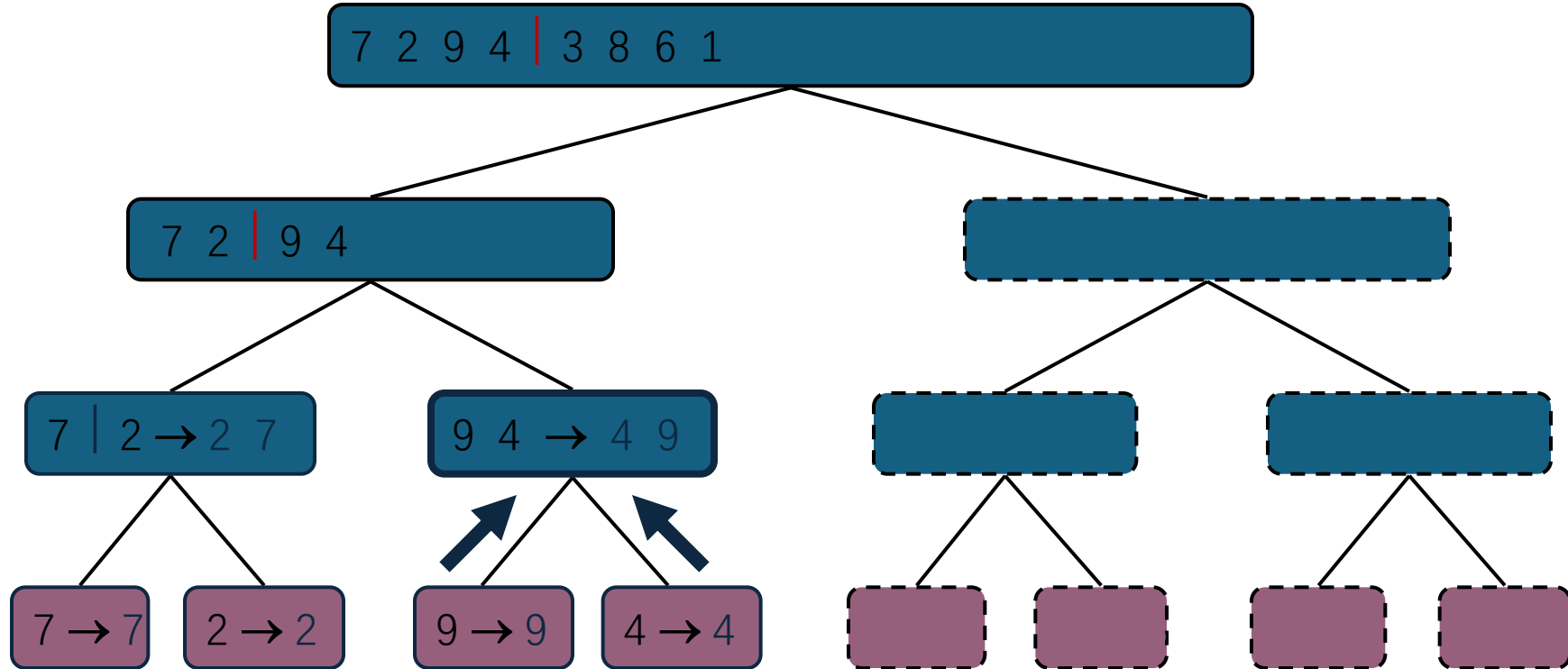
An Example (6)

- Merge two sequences into a longer sorted sequence.



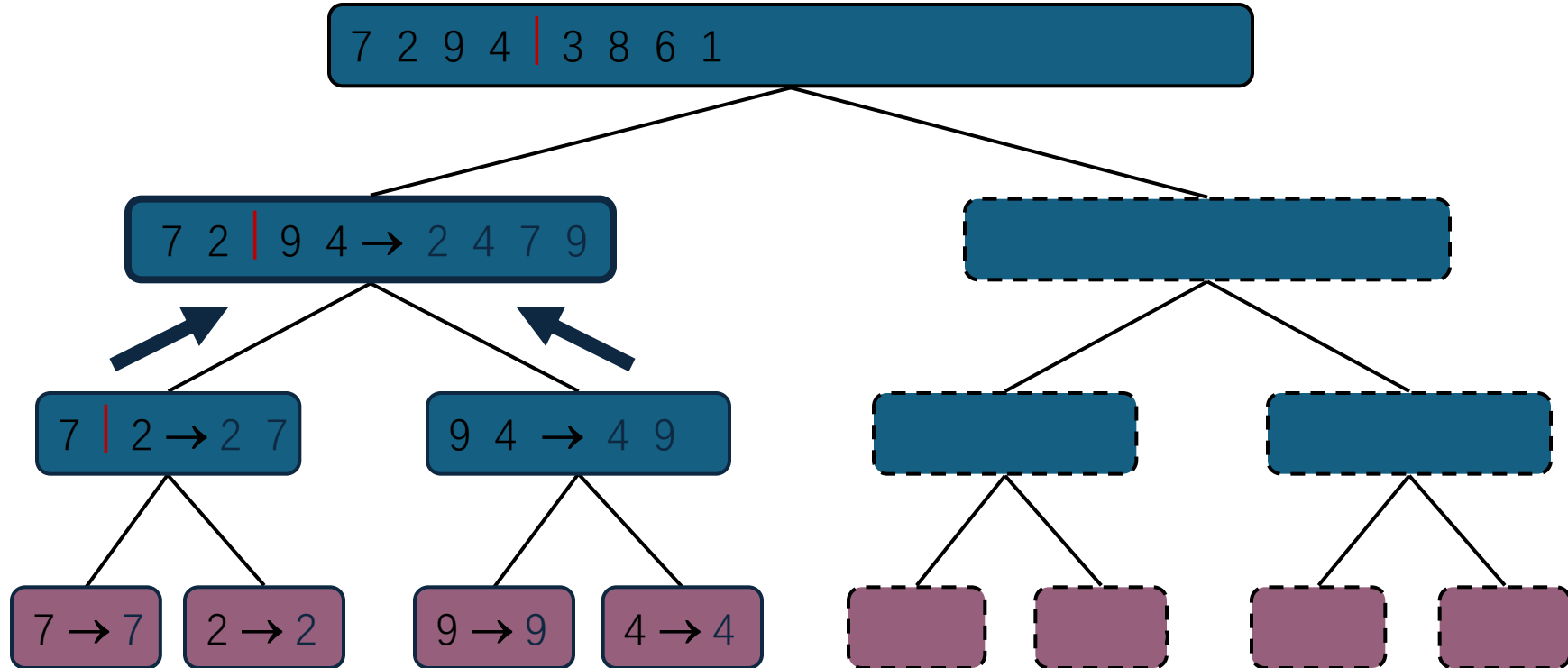
An Example (7)

- Recursive call, ..., base case, merge



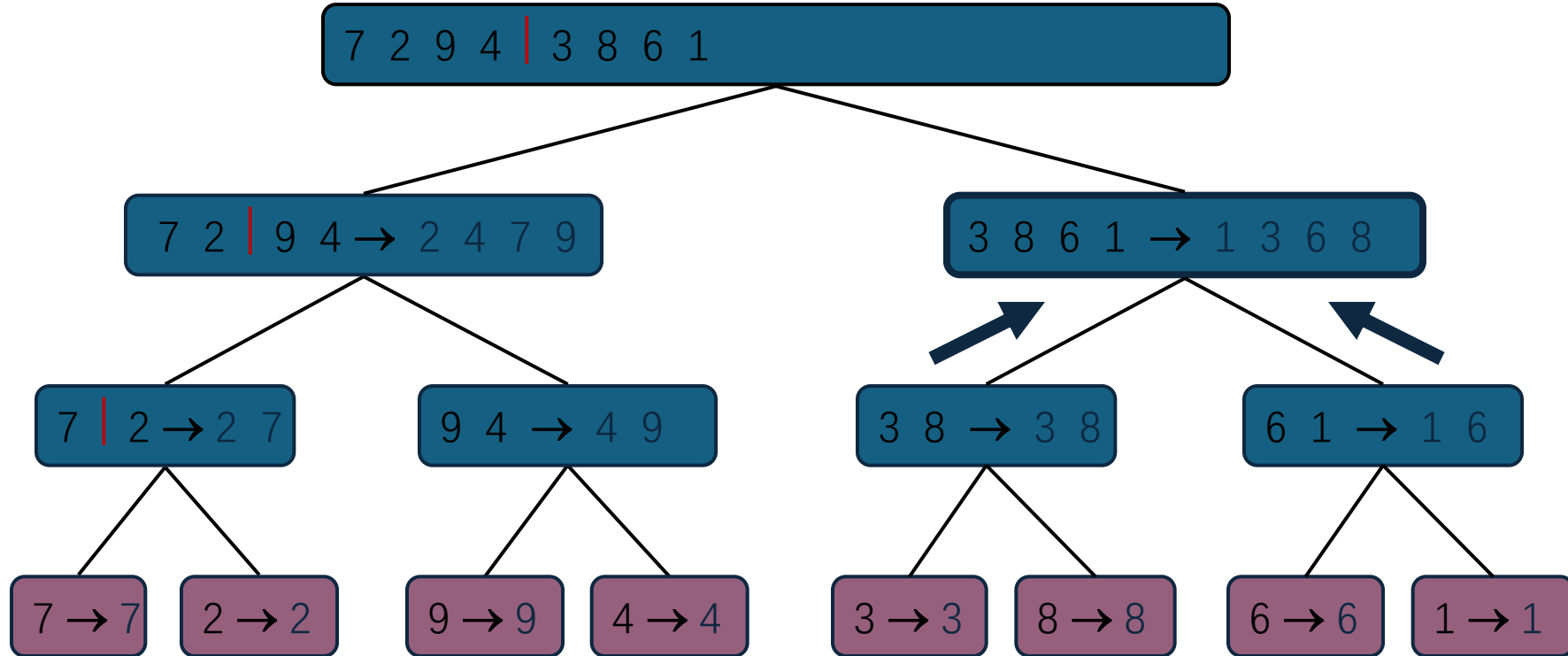
An Example (8)

- Merge two sequences into a longer sorted sequence.



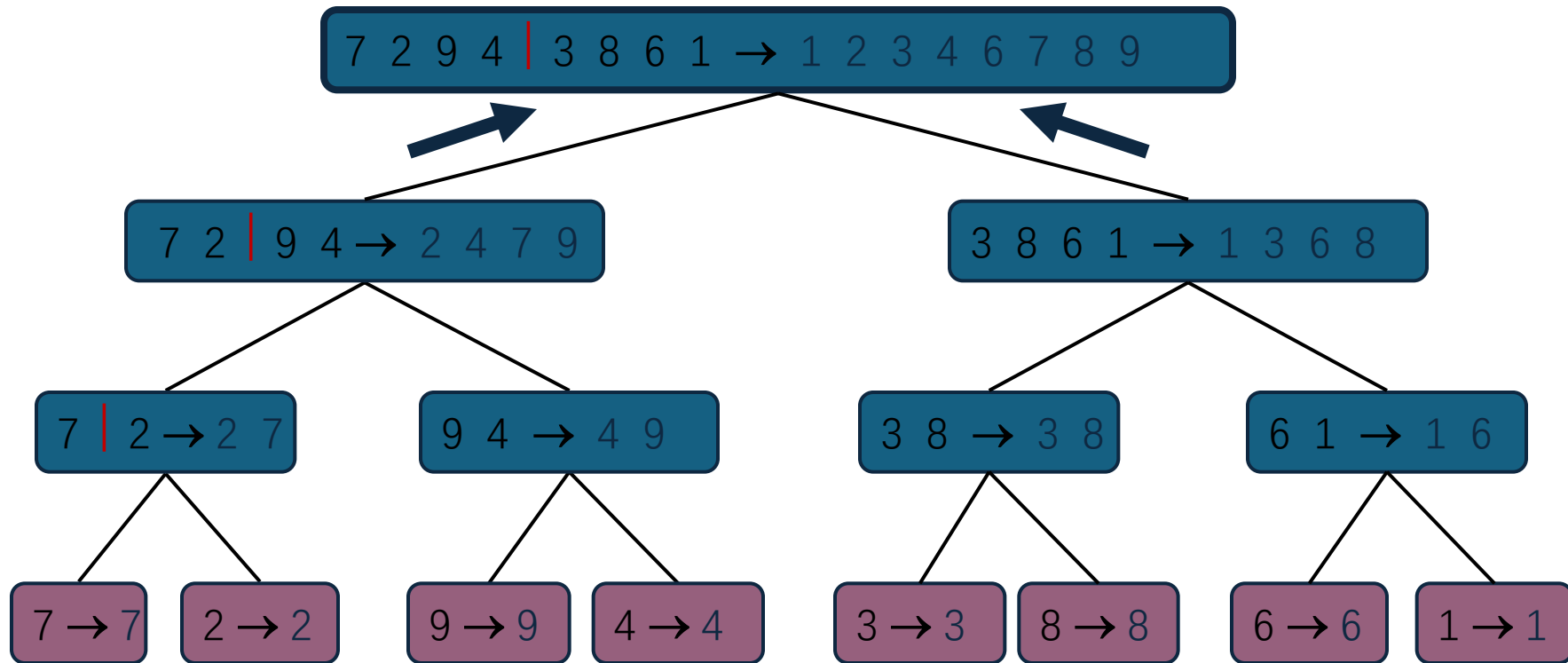
An Example (9)

- Recursive call, ..., merge, merge



An Example (10)

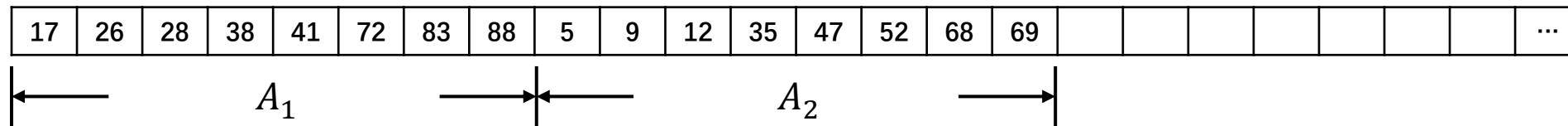
- Merge two sequences into a longer sorted sequence.



The Merging Problem

- There are two arrays — denoted as A_1 and A_2 — each containing (at most) $n/2$ integers in ascending order. The goal is to produce a sorted array A containing all the integers in A_1 and A_2 .

The following shows an example of the input:



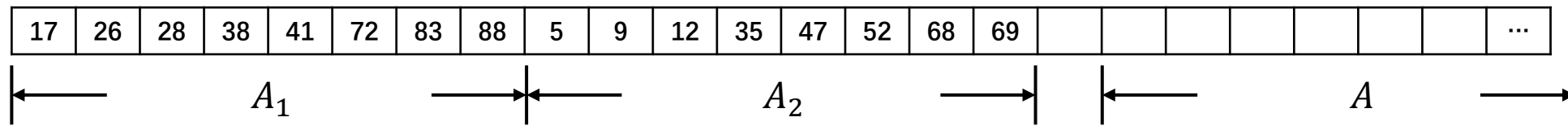
The Merging Problem

- At the beginning, set $i = j = 1$
- Repeat until $i > n/2$ or $j > n/2$:
 1. If $A_1[i]$ (i.e., the i -th integer of A_1) is smaller than $A_2[j]$, append $A_1[i]$ to A , and increase i by 1.
 2. Otherwise, append $A_2[j]$ to A , and increase j by 1.

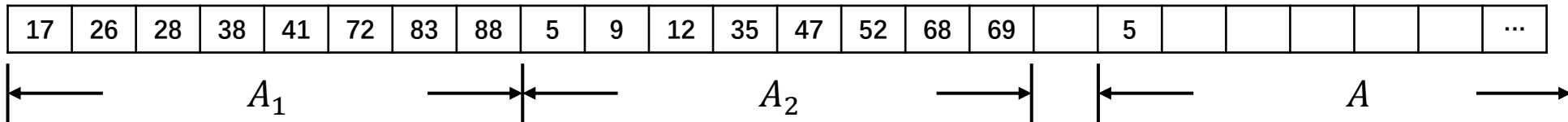
Think: What happens if $i > n/2$? What will you do to complete the merging?

The Merging Problem: Example

- At the beginning of merging:

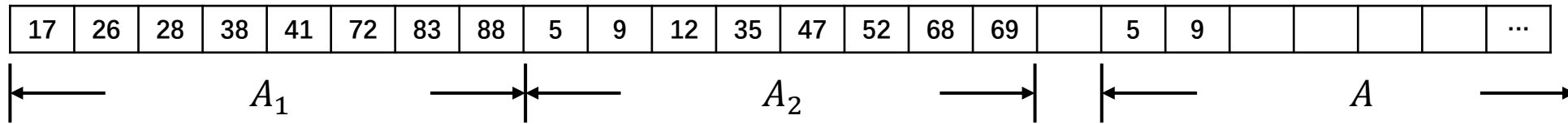


- Appending 5 to A:

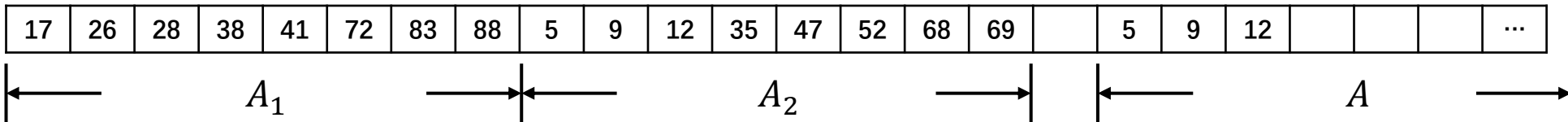


The Merging Problem: Example

- Appending 9 to A:

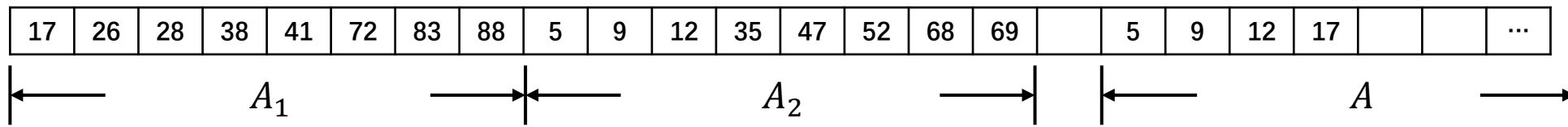


- Appending 12 to A:



The Merging Problem: Example

- Appending 17 to A:



- And so on:

Exercise 1 - Trace the Full Process

- Apply **Merge Sort** to the array:

[8, 3, 7, 4]

- Fill in the steps.

Divide level 1:

[___ , ___] [___ , ___]

Divide level 2:

[___] [___] [___] [___]

Merge step 1:

[___ , ___] [___ , ___]

Final merge:

[___ , ___ , ___ , ___]

Exercise 2 — Count Comparisons

- When merging the arrays

[4, 8, 12]

[3, 7, 9]

How many **comparisons** are needed?

Exercise 4 — Recursion Depth

- For an array of size **16**, how many **levels of recursion** will Merge Sort produce?

The Merging Problem: Running Time

- Let $f(n)$ denote the worst-case running time of merge sort when executed on an array of size n .

- For $n = 1$, we have:

$$f(n) = O(1)$$

- For $n \geq 1$, we have:

$$f(n) \leq 2 \left(f \left\lceil \frac{n}{2} \right\rceil \right) + O(n)$$

where the term $2 \left(f \left\lceil \frac{n}{2} \right\rceil \right)$ is because the recursion sorts two arrays each of size at most $\left\lceil \frac{n}{2} \right\rceil$, and the term $O(n)$ is the time of merging.

The Merging Problem: Running Time

- Suppose n is a power of 2.
- It remains to solve the following recurrence:

$$\begin{aligned}f(1) &\leq c_1 \\f(n) &\leq 2 f(n/2) + c_2 n\end{aligned}$$

where c_1, c_2 are constants (whose values we do not care).

The Merging Problem: Running Time

- So it remains to solve the following recurrence:

$$\begin{aligned}f(1) &\leq c_1 \\f(n) &\leq 2 f(n/2) + c_2 n\end{aligned}$$

- We have:

$$\begin{aligned}f(n) &\leq 2f(n/2) + c_2 n \\&\leq 2(2f\left(\frac{n}{4}\right) + c_2 n/2) + c_2 n = 4f\left(\frac{n}{4}\right) + 2c_2 n \\&\leq 4(2f\left(\frac{n}{8}\right) + c_2 n/4) + 2c_2 n = 8f\left(\frac{n}{8}\right) + 3c_2 n \\&\dots \\&\leq 2^i f\left(\frac{n}{2^i}\right) + i \cdot c_2 n \\&\dots \\&\leq 2^h f(1) + h \cdot c_2 n \quad (h = \log_2 n) \\&\leq n \cdot c_1 + c_2 n \cdot \log_2 n = O(n \log n).\end{aligned}$$

The Merging Problem: Running Time

- How to remove the assumption that n is a power of 2?
- Hint: The rounding approach discussed in a previous lecture.

Exercise 5 — Recursion Tree Work

- Complete the work (operations) per level for Merge Sort.

Level 0: size $n \rightarrow$ work = ____

Level 1: two problems size $n/2 \rightarrow$ work = ____

Level 2: four problems size $n/4 \rightarrow$ work = ____

Exercise 6 — Nearly Sorted Input

- Consider the array

[1, 2, 3, 4, 5]

- Does Merge Sort run faster than normal?

Exercise 7 — Modify Merge Sort to Count Inversions

- An **inversion** in an array is a pair of indices (i,j) such that:

- $i < j$
- $A[i] > A[j]$

- For example, in:

- $[3,1,2]$

- The inversions are:

- $(3,1)$
- $(3,2)$

- So the total number of inversions is **2**.

Now, modify **Merge Sort** so that it not only sorts the array, but also **counts the number of inversions**.
and give the total inversion count of following array

$[2,4,1,3,5]$

How Is Data Sorted?

- Data can be in order before sorting.
 - Random
 - Nearly sorted
 - Reverse
 - Few unique
- See <http://www.sorting-algorithms.com/> for animation.